

Week 8: Digital Twinning & Calibration

EE0849 Introduction to Robotics - Detailed Notes

Basri Erdogan

Table of contents

1	Introduction: The Reality Gap	2
1.1	Why the Gap Matters	2
1.2	Chapter Overview	2
2	Accuracy vs. Repeatability	2
2.1	Definitions	2
2.2	Typical Values	2
2.3	Industrial Implication	3
2.4	Worked Example	3
3	Sources of Positioning Error	3
3.1	Geometric Errors	4
3.2	Gear Backlash	4
3.3	Compliance (Structural Deflection)	4
3.4	Thermal Effects	5
3.5	Servo Errors	5
3.6	Error Summary	5
4	Calibration Methods	5
4.1	Mastering (Joint Zero Calibration)	5
4.2	TCP Calibration (4-Point Method)	6
4.3	User Frame Calibration (3-Point Method)	6
4.4	Full Kinematic Calibration	7
5	FANUC World Zones and Motion Limits	7
5.1	Joint Limits (Soft Limits)	7
5.2	Cartesian Limits (World Zones)	7
6	Advanced TP Programming	8
6.1	Position Registers (PR[])	8
6.2	Offset Positions	8
6.3	User Frames (UFRAME)	9
6.4	Tool Frames (UTOOL)	9
6.5	Multi-Program Architecture	10
6.6	Palletizing with Offsets	10
7	Summary	11
7.1	Key Concepts	11
7.2	Best Practices	11
7.3	Common Mistakes	11

1 Introduction: The Reality Gap

Throughout Weeks 1–5 we built a mathematical model of the FANUC LR Mate 200iD in Python. We assigned DH parameters, solved forward and inverse kinematics, and commanded the virtual robot to reach any reachable pose with perfect precision. In Weeks 6–8 we moved to the physical robot and discovered that reality is messier than mathematics.

This chapter bridges the two worlds. We study *why* the physical robot does not land exactly where the model predicts, *how much* the deviation typically is, and *what we can do about it*.

1.1 Why the Gap Matters

Consider a simple experiment: compute joint angles in Python to place the TCP at (400, 200, 300) mm, then command the real FANUC with those same angles. The TCP might arrive at (399.2, 200.8, 299.5) mm — roughly 1.5 mm off. For a pick-and-place task with 5 mm clearance, this is tolerable. For a pin-insertion task with 0.1 mm tolerance, it is not.

Understanding the sources and magnitudes of this error is the first step toward managing it.

1.2 Chapter Overview

1. Accuracy vs. repeatability
2. Sources of positioning error
3. Calibration methods
4. World zones and motion limits
5. Advanced TP programming (position registers, offsets, frames)

2 Accuracy vs. Repeatability

These two specifications are the most important numbers on a robot’s data sheet, yet they measure fundamentally different things.

2.1 Definitions

Accuracy is the distance between the *commanded* position and the *actual* position the robot reaches:

$$\text{Accuracy} = \|\mathbf{p}_{\text{actual}} - \mathbf{p}_{\text{commanded}}\|$$

It tells you how well the robot’s internal model matches physical reality. A robot with ± 1 mm accuracy can be expected to land within a 2 mm diameter sphere around any commanded Cartesian point.

Repeatability is the spread of *actual* positions when the robot returns to the *same* taught point many times:

$$\text{Repeatability} = 3\sigma(\{\mathbf{p}_i\}_{i=1}^N)$$

where σ is the standard deviation of the position samples. Repeatability tells you how consistent the mechanism is, regardless of whether the mean position is correct.

2.2 Typical Values

Specification	Typical Range	Meaning
Accuracy	$\pm 0.5 - \pm 2.0$ mm	Distance from commanded point
Repeatability	$\pm 0.02 - \pm 0.08$ mm	Spread around the mean

Specification	Typical Range	Meaning
---------------	---------------	---------

Repeatability is 10–100× better than accuracy for most industrial robots. This asymmetry drives a fundamental design choice in industry.

2.3 Industrial Implication

Because repeatability is so much better than accuracy, industrial applications favour **online teaching** over **offline programming** wherever possible:

Online teaching (exploits repeatability):

1. Jog the robot manually to the desired position.
2. Record the joint encoder values.
3. During production, the robot returns to those exact encoder values.
4. The TCP lands in the same physical spot every time (within ± 0.02 – 0.08 mm).

Offline programming (depends on accuracy):

1. Compute joint angles from a CAD model or kinematic solver.
2. Command the robot to those calculated angles.
3. The TCP lands near the intended point, but offset by the accuracy error (± 0.5 – 2 mm).

Offline programming is necessary when teaching is impractical (large or complex parts, hazardous environments, frequent changeovers). FANUC’s offline programming environment, **Roboguide**, serves as the digital twin for this purpose — it replicates the robot’s kinematics, joint limits, and tool definitions in simulation. However, the accuracy of programs developed in Roboguide depends on how well the virtual model matches the physical robot, which is why calibration is essential.

2.4 Worked Example

Suppose you command a FANUC to position (500.0, 0.0, 400.0) mm ten times:

Trial	X (mm)	Y (mm)	Z (mm)
1	499.3	0.6	399.5
2	499.4	0.5	399.6
3	499.2	0.7	399.4
...	...	...	...
Mean	499.3	0.6	399.5

$$\text{Accuracy} = \sqrt{(500 - 499.3)^2 + (0 - 0.6)^2 + (400 - 399.5)^2} = \sqrt{0.49 + 0.36 + 0.25} \approx 1.05 \text{ mm.}$$

$$\text{Repeatability} = 3\sigma \text{ of the spread around the mean} \approx \pm 0.05 \text{ mm.}$$

The robot consistently goes to the *wrong* place (accuracy error), but it goes to *almost the same* wrong place every time (good repeatability).

3 Sources of Positioning Error

Five categories of error contribute to the accuracy gap.

3.1 Geometric Errors

Manufacturing tolerances mean the physical link lengths, joint offsets, and axis alignments differ from the nominal DH model:

Parameter	Nominal	Typical Tolerance
Link length	400.00 mm	± 0.05 mm
Joint offset	0.00°	$\pm 0.01^\circ$
Axis alignment	90.00°	$\pm 0.02^\circ$

Each small error propagates through the kinematic chain. By the time we reach the TCP (after 6 joints), the cumulative effect can reach 0.1–1 mm.

Geometric errors are **constant** for a given robot and can be calibrated out by identifying the true DH parameters.

3.2 Gear Backlash

Every gear train has a small dead zone — the “play” between meshing teeth. When a joint reverses direction, the motor must rotate through this dead zone before the link begins to move.

Effect on positioning:

- The TCP position depends on the *direction of approach*.
- Going clockwise to 45° and counter-clockwise to 45° may yield different positions.
- Typical backlash: 0.5–2 arc-minutes (0.008° – 0.033°) per joint.

For a link of length L and backlash angle $\Delta\theta$:

$$\Delta x \approx L \cdot \sin(\Delta\theta) \approx L \cdot \Delta\theta$$

A 400 mm link with 1 arc-minute backlash produces $400 \times 2.9 \times 10^{-4} \approx 0.12$ mm position uncertainty.

Mitigation: Always approach a target position from the same direction. Spring-preloaded gearboxes reduce backlash mechanically.

3.3 Compliance (Structural Deflection)

The robot arm is not infinitely rigid. Under load, links and joints deflect:

$$\delta = \frac{F \cdot L^3}{3EI}$$

where F is the applied force, L is the effective beam length, E is the elastic modulus, and I is the second moment of area.

Characteristics:

- **Pose-dependent:** Deflection is worst at full arm extension.
- **Load-dependent:** Heavier payloads cause more bending.
- **Gravity-dependent:** Even the arm’s own weight deflects the structure.

For a FANUC LR Mate at full extension with a 5 kg payload, deflection can be 0.5–1.5 mm. At a tucked-in pose near the base, deflection may be under 0.1 mm.

Mitigation: Compliance compensation (built into some controllers), or avoid full-extension poses for precision tasks.

3.4 Thermal Effects

Metal expands when heated. The coefficient of linear thermal expansion gives:

$$\Delta L = L \cdot \alpha \cdot \Delta T$$

Material	α ($\times 10^{-6}$ /°C)
Steel	12
Aluminum	23

Example: A 1 m aluminum structural element warming by 10°C expands by $1000 \times 23 \times 10^{-6} \times 10 = 0.23$ mm.

After power-on, motor heat conducts into the arm structure. It can take 30–60 minutes for the robot to reach thermal equilibrium. During this warm-up period, positions drift.

Mitigation: Run the robot through its motion cycle for 15–30 minutes before precision work. Some applications re-teach critical positions after warm-up.

3.5 Servo Errors

The joint position control loop is not perfect:

- **Following error:** During motion, the actual joint angle lags behind the commanded angle. Faster motion produces larger following error.
- **Settling time:** After a FINE-terminated move, the joint oscillates briefly before settling. Typical settling time is 50–100 ms.
- **Servo stiffness:** Higher servo gain reduces steady-state error but can introduce vibration.

Servo errors are typically the smallest contributor (0.01–0.1 mm) and are mostly handled by the controller’s tuning parameters.

3.6 Error Summary

Error Source	Magnitude	Constant?	Correctable?
Geometric	0.1–1 mm	Yes	Yes (calibration)
Gear backlash	0.05–0.2 mm	Direction-dependent	Partial (approach direction)
Compliance	0.5–3 mm	Pose/load-dependent	Yes (compensation)
Thermal	0.1–0.5 mm	Time-dependent	Yes (warm-up)
Servo	0.01–0.1 mm	Speed-dependent	Yes (tuning)

4 Calibration Methods

Calibration is the process of measuring and correcting the robot’s internal model to reduce accuracy error.

4.1 Mastering (Joint Zero Calibration)

Purpose: Define the encoder zero position for each joint.

Every joint has an absolute encoder or resolver that must know where “zero degrees” is. If this reference is lost (after encoder replacement, collision, or battery failure), the robot’s model is offset by an unknown angle on one or more joints.

FANUC mastering methods:

Method	Description	When Used
Fixture mastering	Align joints with a precision fixture	After motor/encoder replacement
Zero-position mastering	Move to mechanical marks on each joint	Standard re-mastering
Single-axis mastering	Re-master one joint only	After single-axis service
Quick mastering	Use witness marks on joint housings	Quick field re-alignment

After mastering, joint zero errors are typically reduced to under $\pm 0.01^\circ$.

4.2 TCP Calibration (4-Point Method)

Purpose: Find the true location of the Tool Center Point relative to the flange.

When a tool (gripper, welding torch, etc.) is mounted on the flange, the controller needs to know exactly where the functional point (TCP) is. Without this, all Cartesian calculations reference the flange center instead of the tool tip.

Procedure:

1. Fix a sharp reference pin in the workspace.
2. Move the robot so the TCP touches the pin tip.
3. Record the flange pose.
4. Approach the same pin from 3 more different orientations (at least 4 total).
5. The controller solves for the TCP offset (dx, dy, dz) that makes all four flange poses consistent.

Mathematics:

For each approach i , the world position of the TCP must be the same:

$$\mathbf{p}_{pin} = {}^0\mathbf{T}_{flange,i} \cdot {}^{flange}\mathbf{p}_{tcp}$$

This gives us a system of equations. With 4 or more approaches, we solve by least squares for ${}^{flange}\mathbf{p}_{tcp}$.

Accuracy: A careful 4-point calibration typically achieves TCP accuracy of ± 0.5 mm. The 6-point method (which also finds orientation) is more precise.

4.3 User Frame Calibration (3-Point Method)

Purpose: Define a coordinate system attached to a workpiece or fixture.

When positions are taught relative to a user frame, moving the fixture only requires re-calibrating the user frame (3 touch points). All taught positions automatically update.

Procedure:

1. **Origin point:** Touch the corner of the fixture.
2. **X-direction point:** Touch a point along the desired X axis.
3. **Y-plane point:** Touch a point roughly along the Y axis (in the XY plane).

The controller computes the full ${}^{World}T_{User}$ transform using:

- Origin defines the translation.
- X-direction minus origin defines $\hat{x}$.
- Y-plane point, projected to be orthogonal to $\hat{x}$, defines $\hat{y}$.
- $\hat{z} = \hat{x} \times \hat{y}$ completes the right-hand frame.

Benefits:

- Programs become portable: move the fixture, re-calibrate the frame, done.
- Multiple stations can share the same program with different user frames.
- Easier to reason about positions in fixture coordinates than world coordinates.

4.4 Full Kinematic Calibration

Purpose: Identify the true DH parameters of the robot for sub-millimeter accuracy.

This is the most comprehensive (and expensive) calibration method. It requires an external measurement system — typically a laser tracker, photogrammetry system, or coordinate measuring machine (CMM).

Procedure:

1. Move the robot to 50–100+ poses spanning the workspace.
2. At each pose, measure the true TCP position with the external system.
3. Formulate an optimization problem: find the DH parameters that minimize the residual between the model-predicted and measured TCP positions.
4. Update the controller’s kinematic model with the identified parameters.

Achieves: Accuracy of ± 0.1 – 0.3 mm (approaching repeatability).

Cost: Laser tracker equipment costs tens of thousands of euros, and the calibration process takes several hours. This is typically done once during commissioning or after major maintenance.

Once full kinematic calibration is performed, the corrected parameters can be loaded into Roboguide so that the digital twin matches the physical robot to sub-millimeter accuracy.

5 FANUC World Zones and Motion Limits

5.1 Joint Limits (Soft Limits)

Each joint has software-enforced position limits:

Joint	Lower Limit	Upper Limit	Range
J1	-170°	$+170^\circ$	340°
J2	-135°	$+70^\circ$	205°
J3	-80°	$+270^\circ$	350°
J4	-190°	$+190^\circ$	380°
J5	-125°	$+125^\circ$	250°
J6	-360°	$+360^\circ$	720°

These are *software* limits that can be adjusted within the *hardware* (mechanical) limits. If the robot attempts to move past a soft limit, the controller stops motion and raises an error.

5.2 Cartesian Limits (World Zones)

World zones define regions in Cartesian space where the robot’s TCP is either required to stay inside or forbidden from entering.

Zone types:

Type	Behaviour
Safe Zone	Robot must stay <i>inside</i> ; motion stops if TCP exits
Forbidden Zone	Robot must stay <i>outside</i> ; motion stops if TCP enters
Cartesian Space Interference	Avoid a 3D box; triggers signal or stop
Position Register Interference	Avoid a sphere around a specific position

Type	Behaviour
------	-----------

Configuration example (Forbidden Zone around a fixture):

MENU → SETUP → Frames → Space → Interference

Zone 1:

Type: Cartesian
Shape: Box
X: 400 to 600 mm
Y: -100 to 100 mm
Z: 0 to 200 mm
Action: Stop + Signal D0[10]

Important limitation: Interference checking uses the TCP position only, not the full arm envelope. A link or elbow can still collide with objects even if the TCP is outside the forbidden zone.

6 Advanced TP Programming

6.1 Position Registers (PR[])

Position registers store complete 6-DOF Cartesian poses (X, Y, Z, W, P, R) that any program can read and modify.

PR[] vs. P[]:

Feature	P[] (Local)	PR[] (Global)
Scope	One program only	All programs
Modifiable at runtime	No	Yes
Accessible by index	No	Yes
Typical use	Taught positions	Computed positions

Accessing components:

PR[1,1] ! X component
PR[1,2] ! Y component
PR[1,3] ! Z component
PR[1,4] ! W (Rx) component
PR[1,5] ! P (Ry) component
PR[1,6] ! R (Rz) component

Storing the current position:

PR[1] = LPOS ! Store current Cartesian pose
PR[1] = JPOS ! Store current joint angles

6.2 Offset Positions

Offsets allow computing new positions from existing ones without re-teaching.

Manual offset via PR math:

! Create approach position: 50mm above pick point
PR[2] = PR[1] ! Copy pick position
PR[2,3] = PR[1,3] + 50.0 ! Add 50mm to Z

The OFFSET instruction:

FANUC provides a dedicated offset command:

```
L P[1] 500mm/sec FINE Offset,PR[5]
```

This moves the robot to $P[1]$ shifted by the XYZ values stored in $PR[5]$. The offset is applied in the user frame coordinate system.

Common offset patterns:

Pattern	Offset Axis	Value
Approach from above	Z	+height
Retract after pick/place	Z	+height
Shift along conveyor	X	+distance
Shift to next pallet column	Y	+pitch

6.3 User Frames (UFRAME)

A user frame defines a coordinate system attached to a fixture or workpiece. All positions taught while a user frame is active are stored relative to that frame.

Selecting a user frame in TP:

```
UFRAME_NUM = 1          ! Activate User Frame 1
```

Benefit: If the fixture moves, re-calibrate the user frame with the 3-point method. All taught positions automatically follow because they are stored as offsets from the frame origin, not as absolute world coordinates.

Multiple user frames: A workcell where the same fixture moves between two known positions can use UF[1] and UF[2] (pre-calibrated at each position). The same program runs at both locations by switching UFRAME_NUM before execution — the taught P[] positions are relative to the fixture, so they remain valid in either frame.

Frame consistency: Positions taught under one user frame cannot be executed with a different, unrelated user frame. The FANUC teach pendant enforces this — attempting to run a program with a mismatched UFRAME will generate a frame error rather than allowing the robot to move to an incorrect position. This is a safety feature: the controller validates that the active frame is compatible with the taught positions before executing motion.

6.4 Tool Frames (UTOOL)

A tool frame defines where the TCP is relative to the flange.

Selecting a tool frame in TP:

```
UTOOL_NUM = 1           ! Activate Tool Frame 1
```

Why it matters:

- Without the correct UTOOL, linear moves trace curves at the tool tip (because the controller interpolates around the flange, not the TCP).
- Orientation changes pivot around the flange center instead of the functional tool point.
- Jogging in tool coordinates moves in the wrong directions.

Multiple tools: A robot with a dual gripper (e.g., vacuum on one side, mechanical on the other) can switch between UTOOL[1] and UTOOL[2] within a program.

6.5 Multi-Program Architecture

Production applications use modular program structure:

```
/PROG MAIN
  1: UFRAME_NUM=1 ;
  2: UTOOL_NUM=1 ;
  3: LBL[1] ;
  4: CALL PICK ;
  5: CALL PLACE ;
  6: JMP LBL[1] ;
/END

/PROG PICK
  1: J P[1:Pick_App] 100% FINE ;
  2: L P[2:Pick] 100mm/sec FINE ;
  3: DO[1:Gripper]=ON ;
  4: WAIT .30(sec) ;
  5: L P[1:Pick_App] 500mm/sec FINE ;
/END

/PROG PLACE
  1: J P[1:Place_App] 100% FINE ;
  2: L P[2:Place] 100mm/sec FINE ;
  3: DO[1:Gripper]=OFF ;
  4: WAIT .20(sec) ;
  5: L P[1:Place_App] 500mm/sec FINE ;
/END
```

Advantages:

- Each subprogram is independently testable.
- PICK and PLACE can be reused in different MAIN programs.
- Positions are local to each subprogram (P[1] in PICK is different from P[1] in PLACE).
- Easier to debug: isolate the problem to one subprogram.

6.6 Palletizing with Offsets

A common industrial task: place parts in a grid pattern.

Approach using registers and MOD/DIV:

Given a pallet with C columns and R rows, and a pitch of p mm:

- Column index: $c = n \bmod C$
- Row index: $r = n \div C$ (integer division)
- X offset: $\Delta x = c \cdot p$
- Y offset: $\Delta y = r \cdot p$

```
/PROG PALLETIZE
  1: R[1]=0 ;                ! Part counter
  2: LBL[1] ;
  3: CALL PICK ;
  4: R[2]=R[1] MOD 4 ;       ! Column (0-3)
  5: R[3]=R[1] DIV 4 ;       ! Row (0-2)
  6: PR[10,1]=R[2]*50.0 ;    ! X = col * 50mm pitch
  7: PR[10,2]=R[3]*50.0 ;    ! Y = row * 50mm pitch
  8: PR[10,3]=0 ;
```

```

    9: L P[5:Pallet_Origin] 500mm/sec FINE Offset,PR[10] ;
   10: DO[1:Gripper]=OFF ;
   11: WAIT .20(sec) ;
   12: R[1]=R[1]+1 ;
   13: IF R[1]<12, JMP LBL[1] ; ! 4x3 = 12 parts
/END

```

This places 12 parts in a 4-column $\times$ 3-row grid using a single taught point (the pallet origin) and computed offsets.

7 Summary

7.1 Key Concepts

1. **Repeatability » Accuracy** for industrial robots. Use taught positions whenever possible. Use calibration when offline programming is required.
2. **Five error sources** contribute to the accuracy gap: geometric errors (constant, calibratable), gear backlash (direction-dependent), compliance (pose/load-dependent), thermal expansion (time-dependent), and servo errors (speed-dependent).
3. **Calibration improves accuracy.** Methods range from simple mastering (joint zeros) through TCP and user frame calibration (practical, routine) to full kinematic calibration (comprehensive, expensive).
4. **World zones** protect the robot and its environment through software-enforced Cartesian and joint limits.
5. **Advanced TP programming** with position registers, offsets, user frames, tool frames, and multi-program architecture enables flexible, maintainable automation.

7.2 Best Practices

- **Warm up** the robot before precision work (15–30 minutes).
- **Approach from a consistent direction** to minimize backlash effects.
- **Use FINE termination** at pick and place positions.
- **Calibrate TCP** whenever a tool is mounted or replaced.
- **Use user frames** to make programs portable across fixtures.
- **Structure programs** with MAIN calling task-specific subprograms.

7.3 Common Mistakes

1. Trusting calculated positions for precision tasks without calibration.
2. Skipping warm-up before measuring or teaching critical positions.
3. Assuming accuracy equals repeatability when reading spec sheets.
4. Teaching positions with the wrong UTOOL or UFRAME active.
5. Using global PR[] for positions that should be local P[].
6. Forgetting to set UFRAME_NUM and UTOOL_NUM at the start of a program.